

RISC-V GPU on a Smart NIC



Georgia Tech
Open Source Program Office

Students: Mingjun Xu
Mentors: Eric Lorimer

What is Vortex?

- An open-source RISC-V GPU: a real SIMT processor, can be run on FPGA and program with kernels.

What are we doing?

- Running Vortex on a commodity FPGA SmartNIC (MangoBoost BoostX) – without XRT or any vendor GPU runtime.

What is the goal?

- Boot real GPU test kernels on the board, verify results end-to-end, and scale the config up (memory, warps, threads, FPU).

Three challenges we solved

Finding the data path

- The host's window into GPU memory appeared in none of document. We reverse-engineered with a memory address pork python file and confirmed that data path is : BAR4 + 0x0, 8 MB.

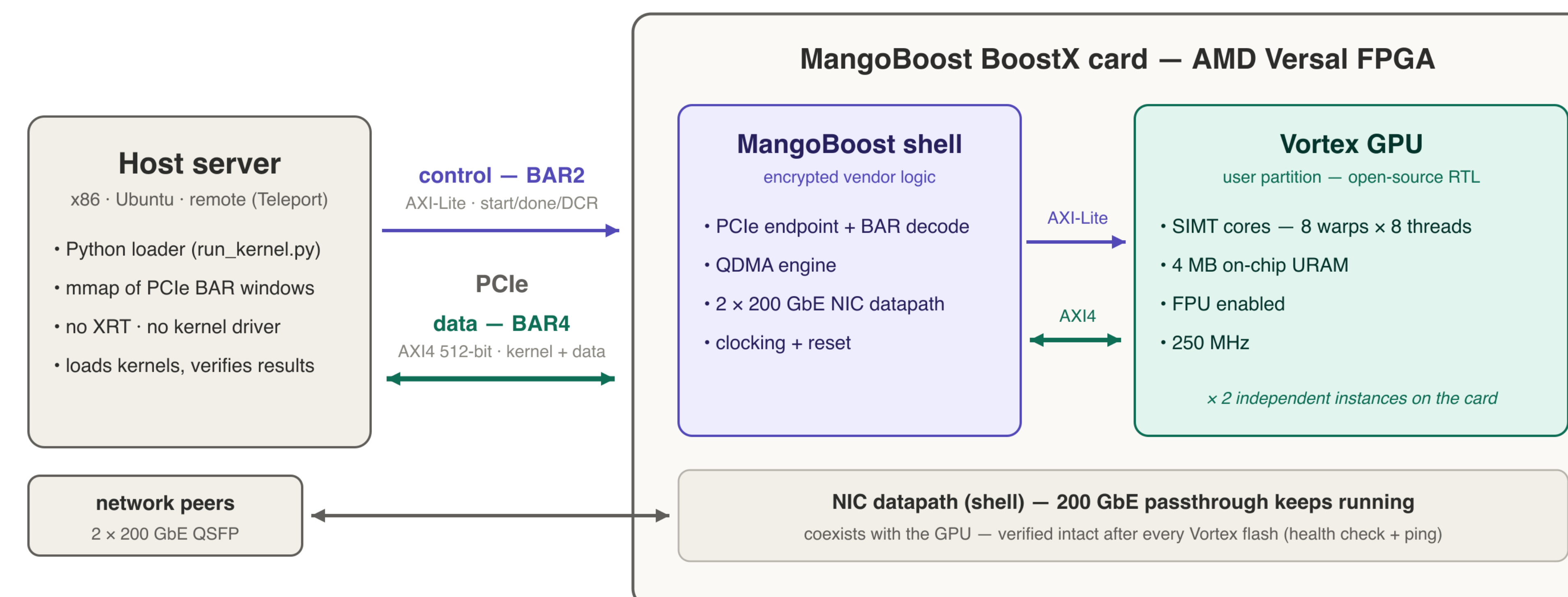
The vanishing build hook

- Vortex needs a Tcl hook before opt_design to patch async-RAM black boxes. Assigning a Vivado strategy silently resets all step hooks – ours vanished, and the build failed 28 black boxes deep.
- The fix: Register hooks after the strategy line.

The FPU that computed wrong answers

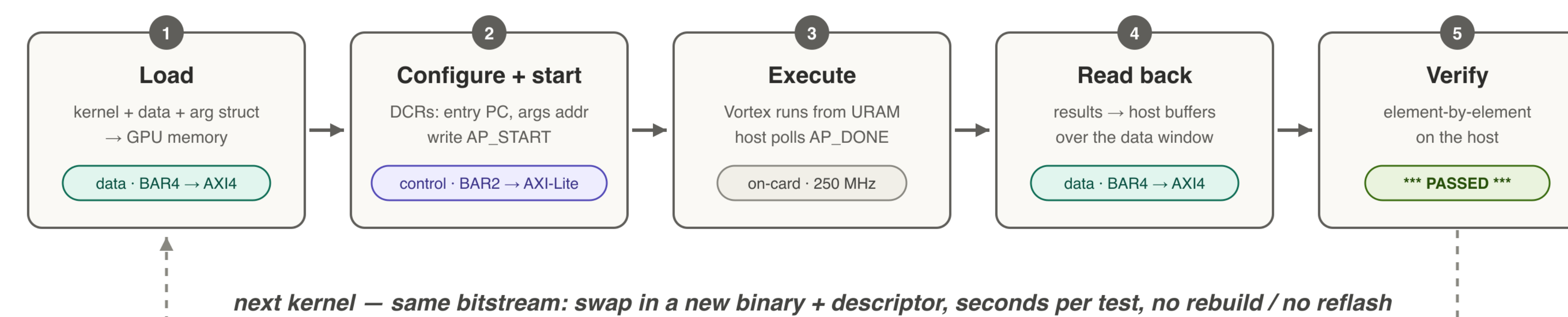
- On Versal, the FPU IP core picks its own pipeline depth – deeper than the latency we had on UltraScale+. Causing timing misalignment and give wrong result.
- The fix: Matching the latency fixed it – ~17 build-flash-reboot cycles to isolate.

Top level and How to Run



How a kernel runs ?

- The loader writes the kernel binary, input data, and the argument struct into GPU memory through BAR4 (AXI4).
- Through BAR2 (AXI-Lite) it sets the DCRs — kernel entry PC and argument address then writes AP_START.



Test Results

```
mxu446@mba-test04:~$ sudo python3 run_kernel.py \
--test sort -n 16384 --ctrl-bar 0x29ffc0000000 --data-addr 0x29ffd0000000
[0] test=sort kernel=sort.bin n=16384
[2] data window write/readback: ok
[3] loaded kernel (29440 B) + ['src'] + args in 0.03s
[5] reset done, DCRs set: PC=0x0 args=0x30000
[6] kernel started
[7] kernel DONE in 3.577s
*** RESULT PASSED (sort, n=16384, 16384 elems, 3.577s) ***

— earlier runs, same bitstream —
*** RESULT PASSED (vecadd, n=262144, 262144 elems, 0.038s) ***
*** RESULT PASSED (diverge, n=16384, 16384 elems, 6.088s) ***

mxu446@mba-test04:~$
```

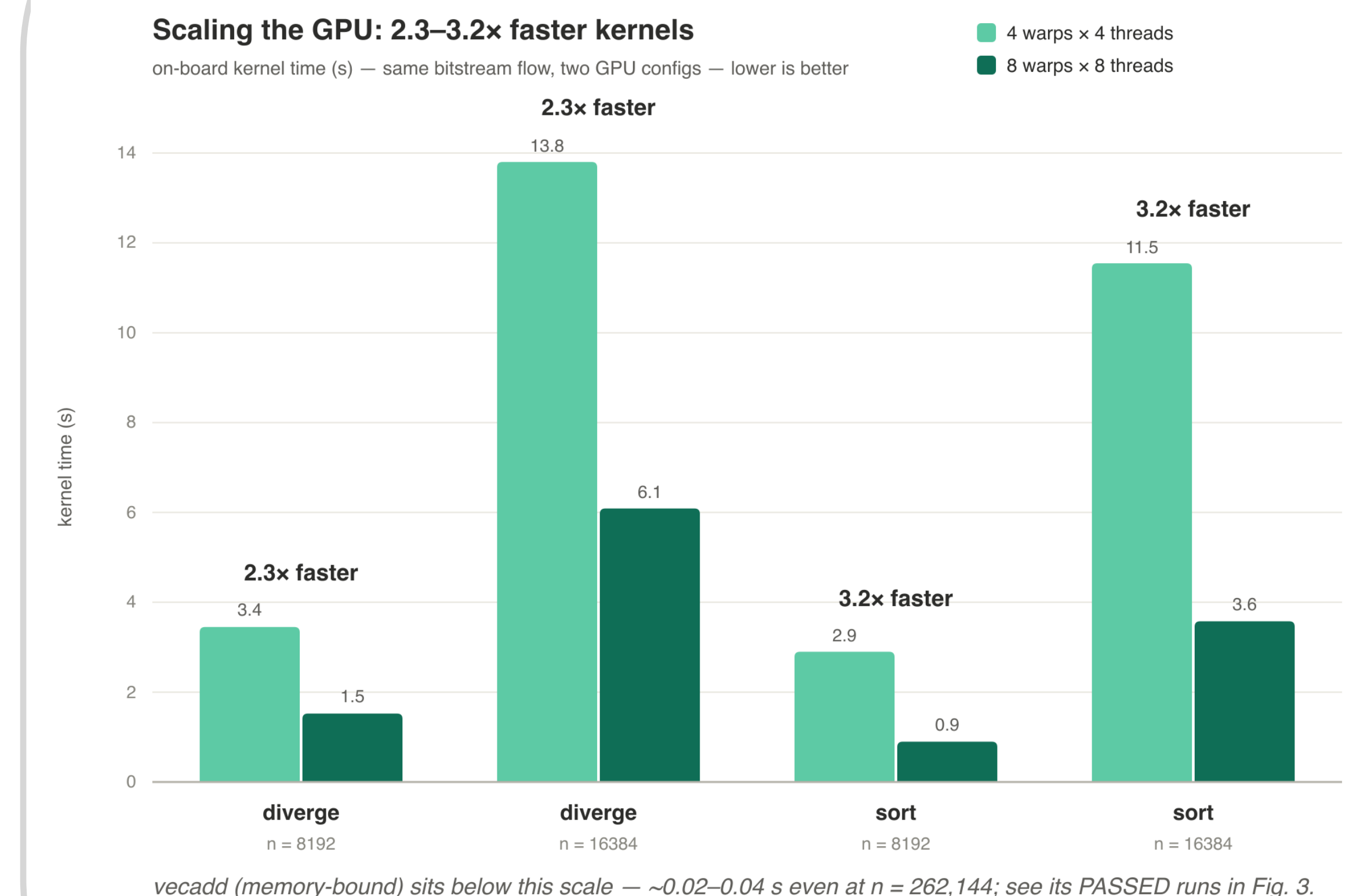
Integer kernels pass:

- vecadd (n = 262 K).
- sort (n = 16 K)
- diverge (n = 16 K, SIMT branch-divergence test).

Float kernel pass:

- vecaddf (passes for small sizes -n < 1000)

Performance



- 8w8t shows great performance improvement.

Future Work

Fix FPU

- Revise Vortex arch. Read result from FPU based on ready signals, not counting cycles.
- Max out warps and threads to achieve more throughput.

Build runtime to not use XRT.

- So future kernels can run directly on Vortex.

Vortex with NIC.

- Vortex data processing right next to NIC; or right on data stream.
- Off load for CPU and save bandwidth.

